

6.3 实验三：模型格式转换与嵌入式部署

6.3.1 实验目的

本实验旨在使学生掌握 YOLOv5 模型从训练产物（best.pt）向嵌入式设备部署格式的转换流程；通过 ONNX 模型中间转换、INT8 量化与硬件适配，最终生成可在 RK3566 平台高效运行的 .rknn 格式模型；学习使用 RKNN-Toolkit2 工具完成模型压缩与优化，并在实际开发板中部署测试，验证边缘端图像识别系统的可行性与推理效果。

6.3.2 实验说明

本实验将完整实现 YOLOv5 模型的端到端部署流程：首先将训练好的 best.pt 模型转换为具有动态输入尺寸支持的 ONNX 格式，随后利用 RKNN-Toolkit2 进行 INT8 量化生成 NPU 专用模型 best.rknn。转换完成后，将部署模型至 RK3566 开发板，结合摄像头进行实际识别测试，验证目标检测模型在边缘设备上的应用可行性。

6.3.3 实验步骤

步骤一：PyTorch 模型转为 ONNX 格式

为实现训练模型在 RK3566 开发板的高效部署，首先将训练得到的 PyTorch 模型（best.pt）转换为 ONNX 中间格式，随后在 Ubuntu 虚拟机环境下调用 rknn-toolkit2 工具包中的 test.py 转换脚本，将 ONNX 模型进一步转化为适配 Rockchip NPU 的 RKNN 格式，最终完成模型在嵌入式平台的轻量化部署，充分发挥硬件加速能力。模型转化流程图如图 6.40 所示：



图 6.40 模型转化流程图

首先，找到实验二中训练完成的 best.py 模型，进入 yolov5-master > run > exp>weights，如图 6.41 所示：

(其中 exp 是你最新一次训练的结果)

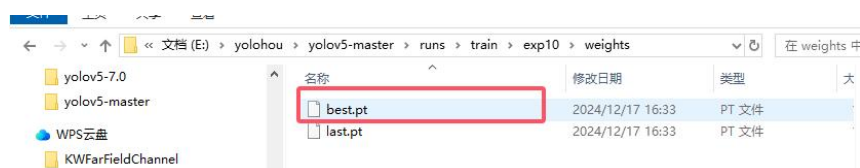


图 6.41 best.py 模型

复制最新的 best.pt 到 yolov5-master 文件下，用 VS code 打开 yolov5-master，找到 export.py，在终端输入 `python export.py --rknpu --weight best.pt`，pt 模型转换 onnx 成功后如图 6.42 所示：

```

PS E:\yolohou\yolov5-master> python export.py --rknpu --weight best.pt
export: data=E:\yolohou\yolov5-master\data\coco128.yaml, weights=['best.pt'], imgsz=[640, 640], device=-1, half=False, inplace=False, keras=False, optimize=False, int8=False, dynamic=False, simplify=True, workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_all=100, iou_threshold=0.45, rknpu=True
YOLOv5 2023-12-19 Python-3.8.20 torch-1.8.2+cu102 CPU

Fusing layers...
Model summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
---> save anchors for RKNPU
[[10.0, 13.0], [16.0, 30.0], [33.0, 23.0], [30.0, 61.0], [62.0, 45.0], [59.0, 119.0], [32.0, 326.0]]
export detect model for RKNPU

PyTorch: starting from best.pt with output shape (1, 18, 80, 80) (13.7 MB)
ONNX: starting export with onnx 1.16.1...
ONNX: export success 1.8s, saved as best.onnx (26.8 MB)

Export complete (聚焦资源管理器中的文件夹 (Ctrl + 单击))
Results saved to E:\yolohou\yolov5-master
Detect: python detect.py --weights best.onnx
Validate: python val.py --weights best.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'best.onnx')
Visualize: https://github.com/ultralytics/yolov5/blob/master/README.md
  
```

图 6.42 pt 模型转换 onnx

如图 6.43 所示。显示以下结果，则说明模型转换成功，在 yolov5-master 文件下找到 best.onnx 文件。

best	2025/12/5 8:58	ONNX 文件
best.pt	2025/2/18 7:17	PT 文件
CONTRIBUTING	2023/12/19 0:00	Markdown 源文
detect	2023/12/19 0:00	Python 源文件
benchmarks	2023/12/19 0:00	Python 源文件
.pre-commit-config	2023/12/19 0:00	YAML 文件
.gitignore	2023/12/19 0:00	Git Ignore 源文
.gitattributes	2023/12/19 0:00	Git Attributes 源文

图 6.43 onnx 模型转换成功

步骤二：ONNX 格式转为 RKNN 格式

打开虚拟机找到 rknn toolkit2-master 包，如图 6.44 所示：

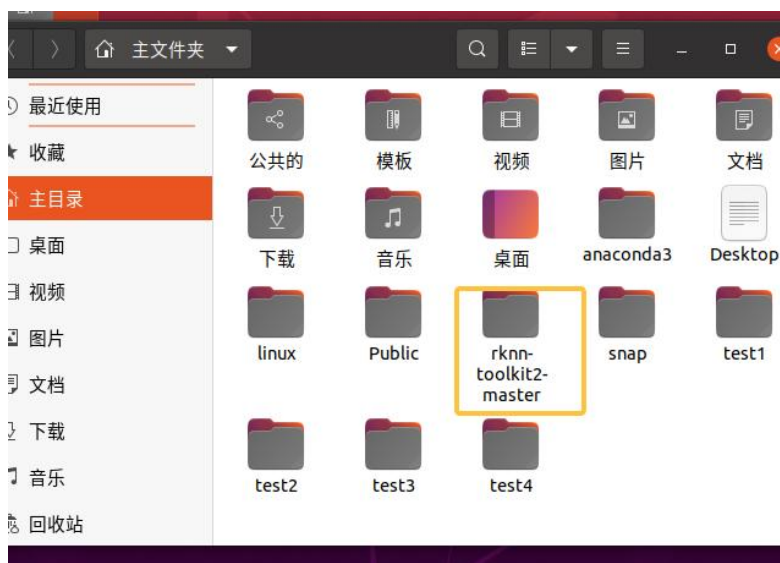


图 6.44 虚拟机 rknn toolkit2-master 包

打开虚拟机，然后依顺序打开 rknn-toolkit2-master->examples->onnx->yolov5，如图 6.45 所示：

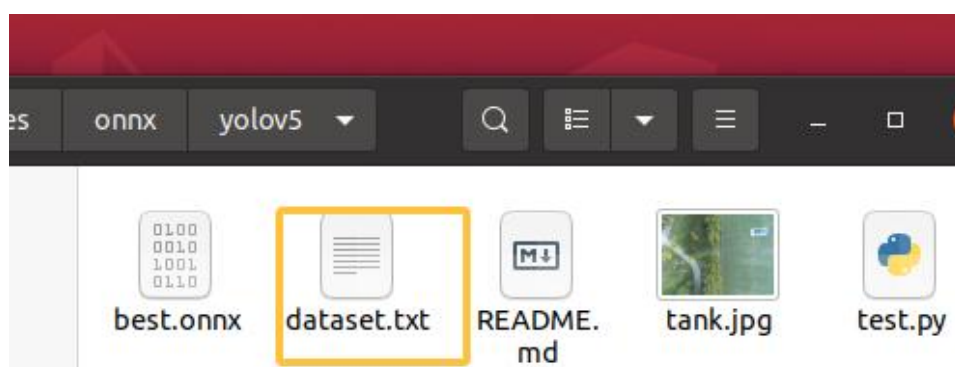


图 6.45 修改 dataset.txt 文件 1

将我们导出的 best.onnx 移入该文件中，打开 dataset.txt，将内容修改为 tank.jpg，如图 6.46 所示：

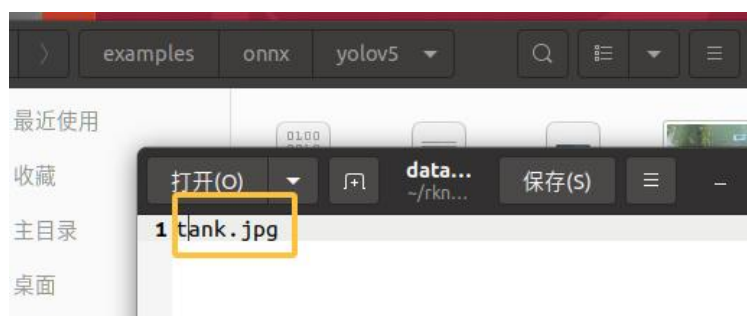


图 6.46 修改 dataset.txt 文件 2

在该路径下打开终端，输入 `conda activate rknn` 激活 rknn 虚拟环境（在实验给出的虚拟机中已经提前安装好 rknn 虚拟环境，想了解的相关步骤可见 https://blog.csdn.net/weixin_71719718/article/details/134454881），在终端中看到（rknn）则说明环境激活成功，输入 `python test.py`，如图 6.47 所示：

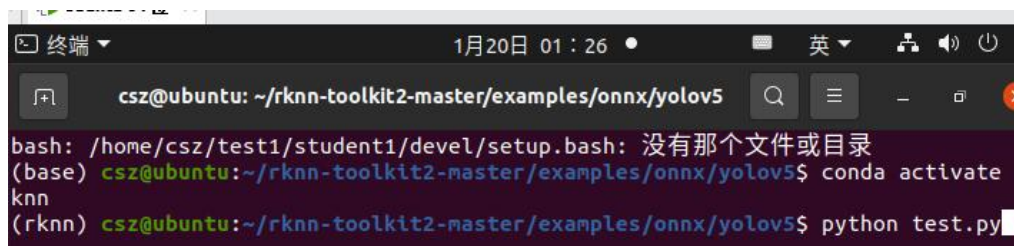


图 6.47 终端转化 RKNN 文件

在终端完成模型转换流程后，目标文件夹中会生成若干输出文件，其中最重要的转换产物是适配 RK3566 平台的.rknn 模型文件，如图 6.48 所示：

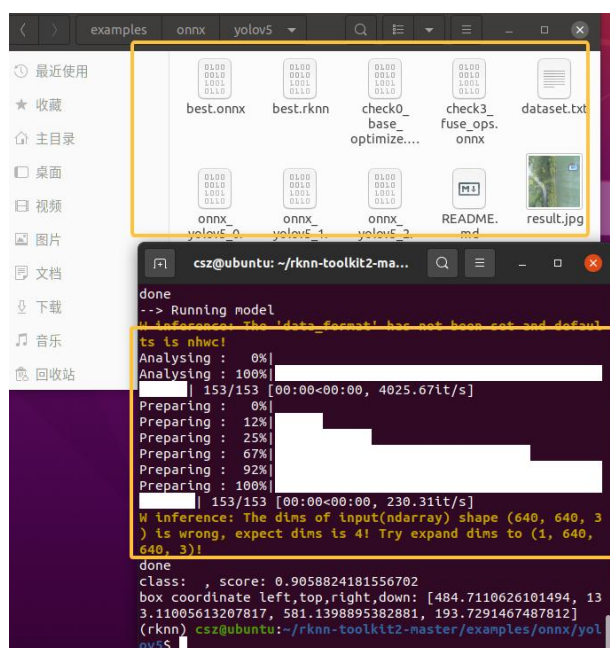


图 6.48 模型转化成功

如图 6.49 所示。通过查看生成的 result.jpg 测试结果图像，可以直观验证转换后的 RKNN 模型在实际场景中的推理效果和准确度表现，这为后续嵌入式部署提供了可靠的性能验证依据。

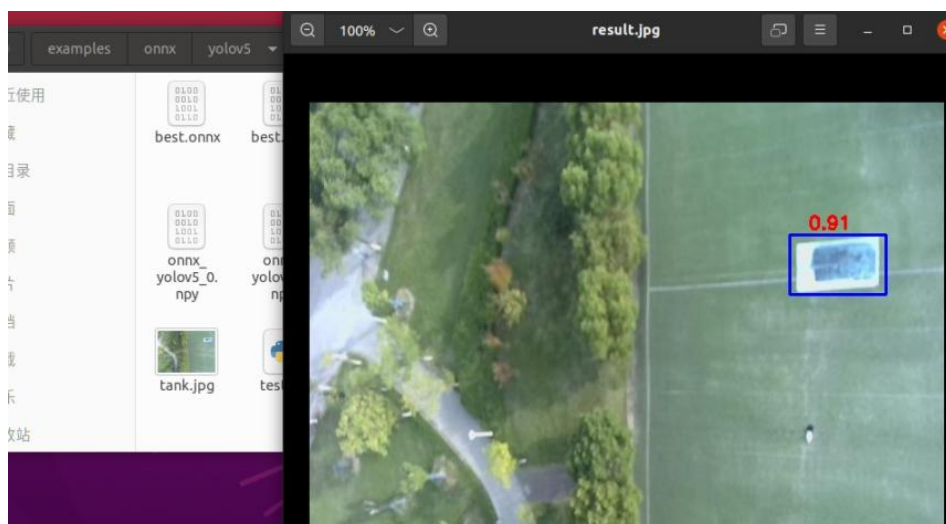


图 6.49 模型转化效果

步骤三：将模型部署在 RK3566

打开 MobaXterm，在泰山派 RK3566 上部署已转换为.rknn 格式的 YOLOv5s 模型，路径和命名如图 6.50 所示（可对提供的文件进行替换）：

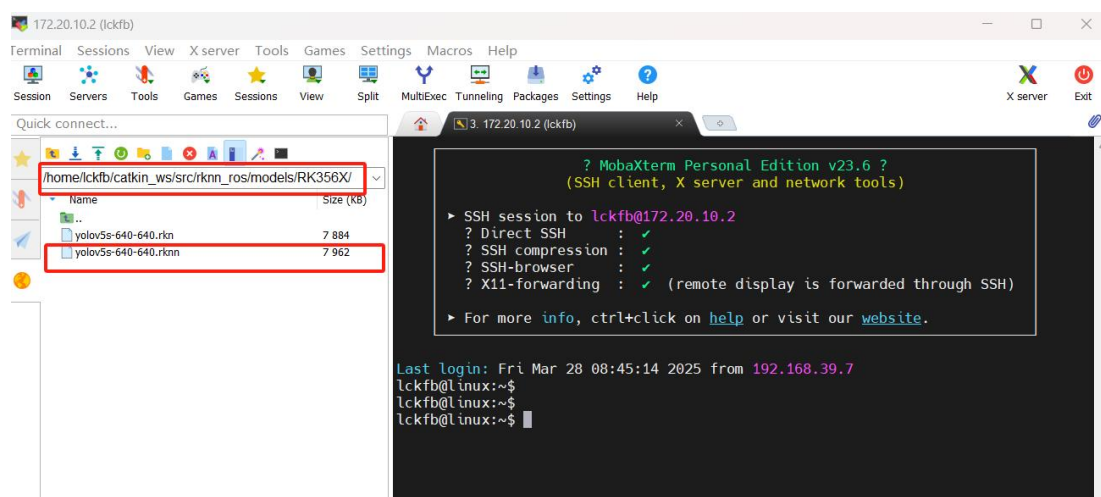


图 6.50 模型部署

步骤二：测试模型，RK3566 连接摄像头，启动摄像头节点 `roslaunch rknn_ros camera.launch device:=video0`（可通过 `ls /dev/video*` 查看可用的视频设备），另起一个终端，启动 yolo 识别 `roslaunch rknn_ros yolov5.launch chip_type:=RK356X`，在此可实时查看到是否识别到目标以及识别的精度。可另开一个终端，输入 `rqt_image_view`，接收图像数据，图像化显示识别效果。可在打开的窗口选择 `/rknn_image/theora` 显示识别后的效果，，如图 6.51 所示：

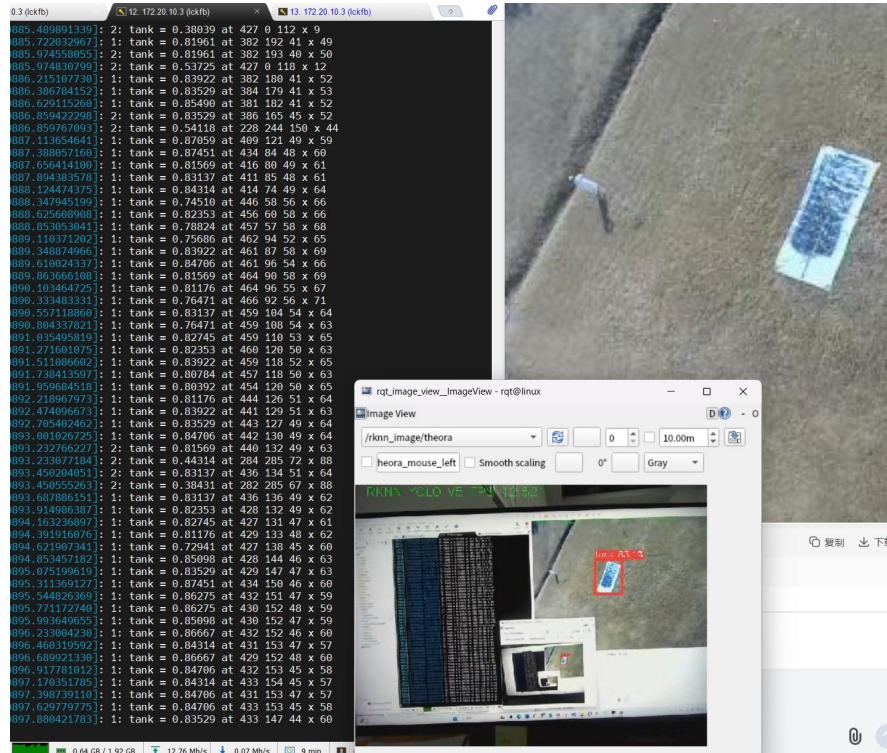


图 6.51 查看模型识别效果